MATH 4162/Math 6201
Numerical methods for differential equations
Fall 2025

Case Study #2

Show all necessary steps of working in the given space

Name:——————————————————-

ID:——————————————————-

Upload the assignment solution no later than

23:59 on Nov 25, 2025

Full marks 50

You will submit the solution code through a Jupyter notebook (or any other form of your choice). Each question may require coding and texts for some explanation. You may write your solution/explanation through mark-down cell or write in a separate paper.

Upload your code and solution through BrightSpace, Assignment.

If you need coding help, see `https://github.com/alamj/DataDrivenFluid`

1. (a) Given $u(x) = \sin(2\pi x)$ for $x \in [0, 1]$ and consider the Fourier series

$$u(x) = \sum_{k=-\infty}^{k=+\infty} \hat{u}_k e^{-ikx},$$

where $\hat{u}_k$ is the Fourier transform of $u(x)$.

Interpret why or how

$$\left\langle \frac{\partial u}{\partial x}, e^{ikx} \right\rangle = -ik\hat{u}_k \quad \text{and} \quad \left\langle \frac{\partial^2 u}{\partial x^2}, e^{ikx} \right\rangle = -k^2\hat{u}_k$$

(i.e. Fourier transforms of $u_x(x)$ and $u_{xx}(x)$ are $-ik\hat{u}_k$ and $-k^2\hat{u}_k$, respectively).

Calculate u_x and u_{xx} using `numpy.fft` and submit plots. (Notice that numpy uses a opposite sign convention.)

   (b) Consider the viscous Burgers equation in the domain $0 \le x \le 1$:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad 0 < x < 1, \quad t > 0,$$

subject to periodic boundary condition $u(0, t) = u(1, t)$. For the grid $\{x_j\}$ with $x_j = j/n$ and $j = 0, \ldots, n-1$, briefly present/discuss the Euler explicit scheme for solving Burgers equation using Fourier spectral differentiation. This includes writing the correct form of the scheme, stability condition, and how boundary conditions are implemented.

```
def rhsPDE(u,t,kx,nu):
    uhat = np.fft.fft(u)
    d_uhat = (1j)*kx*uhat
    d_u = np.fft.ifft(d_uhat)
    dudt = -u * d_u
    return dudt.real

u = odeint(rhsPDE,u0,t,args=(kx,nu))
```

Figure 1: `rhsPDE` is a template code, which can be used by `scipy's odeint` to solve $u_t + uu_x = 0$. In this case study, you may write your own time integration code using Euler/RK4 or use `odeint/solve_ivp`

For each contour plot, insert a symbol at (t_s, x_s) indicating point of intersection for each case of two ν values.

(c) Using the Fourier spectral differentiation method, calculate $u_x(x,t)$, where $u(x,t)$ the numerical solution of the Burgers equation with $u(x,0) = \sin(2\pi x) + (1/2)\sin(\pi x)$ and $\nu = 0.1$. Find the point (t_r, x_r) where u_x reaches its minimum value. Repeat this to find (t_r, x_r) for $\nu = 0.01,\ 0.001,\ 0.0001$. Plot $t_r - t_s$ vs ν in log-log scale and estimate the logarithmic slope.